Online Appendix: CLVR Ordering of Transactions on AMMs

Anonymous Author(s)

Anonymous Institution(s)

This document is the online appendix to the paper "CLVR Ordering of Transactions on AMMs". Section, figure, and table numbers that do not start with a letter refer to the main paper.

A Other Economics measures

A.1 Inequality

Other economics measurement could be of interests instead of price volatility. In this subsection, we consider inequality. We consider ordering strategy that prioritizes certain traders by organizing trades so that favored participants receive higher prices when selling and lower prices when buying.

Inequality can be evaluated in different ways, depending on the assumptions and objectives of the policymaker. One approach assumes that transaction sizes correlate with users' initial wealth, and orders trades to favor smaller transactions.

Another approach evaluates inequality based on total user wealth, measured through wallet balances. However, this is challenging in the pseudonymous blockchain environment, where users can control multiple accounts to conceal their true wealth (i.e., a *sybil* attack).

Traders' Inequality Based On Their Transaction Size - Gini Post Trade Wealth: For a given set of pending transactions M in a given block, the transaction order that reduces traders' inequality while accounting for transaction size is the one that results in the lowest Gini coefficient (lowest inequality) after the transactions are executed.[1] Intuitively, this works by ensuring that smaller transactions receive a "good" exchange price, while larger transactions receive a "bad" exchange price, thus reducing inequality from both ends.

In a simple example when the exchange rate between token X and token Y is one-to-one ($p_0 = 1$), the amount of tokens each trader redeems from a transaction ($x_{\text{out}}(\sigma, i)$ or $y_{\text{out}}(\sigma, i)$) can represent its "wealth". The "wealth" and corresponding Gini Coeffient are given in the formulas below:

$$W_i = x_{\text{out}}(\sigma,\ j) \text{ or } y_{\text{out}}(\sigma,\ j) \tag{1}$$

$$\textsc{Gini}(\sigma) = \frac{2 \cdot \sum_{i=1}^{n} i \cdot W_i}{n \cdot \sum_{i=1}^{n} W_i} - \frac{n+1}{n} \tag{2}$$

[1] The Gini Coefficient is a well-known economic measurement of inequality. Gini Coefficient ranges from 0 to 1, with higher values indicating greater inequality [27].

Where $W_1 \leq W_2 \leq \cdots \leq W_n$. The social planner will need to minimize the Gini coefficient by ordering the transactions in M in a way that minimizes this function.

Finally, Table S1 shows summary statistics of both price stability and equality objectives from all simulations, which demonstrates that optimizing for one objective produces poor outcomes for the other objective.

Table S1. Relative optimization scores of price stability and equality objectives. Each score is computed as a relative percentage, where 0% is the global minimum, and 100% is the maximum. Taken as the mean of 100 trials. We show the relative Gini coefficient when optimizing for price stability, and likewise the relative volatility when optimizing for equality.

	Number of Trades				
	3	5	10	12	13
	Relative Gini Coefficient				
Maximizing Price Stability	45.02	45.30	48.76	46.93	48.89
Minimizing Price Stability	56.47	55.08	52.25	53.05	51.88
	Relative volatility				
Maximizing Equality	48.53	50.97	52.64	50.71	50.31
Minimizing Equality	47.31	43.07	37.93	37.98	38.05

B Model Equations

The following equations compute the amount received when the ith trade in ordering σ is executed on a constant-product automated market maker with initial reserves x_0 and y_0.

$$y_{\text{out}}(\sigma, i) = \begin{cases} x_{\text{in}} \frac{Y(\sigma,\ i-1)}{X(\sigma,\ i-1)+x_{\text{in}}} & \text{if } \sigma(i) = \text{sell } x_{\text{in}} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

$$x_{\text{out}}(\sigma, i) = \begin{cases} y_{\text{in}} \frac{X(\sigma,\ i-1)}{Y(\sigma,\ i-1)+y_{\text{in}}} & \text{if } \sigma(i) = \text{buy } y_{\text{in}} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

Where functions $X(\sigma,\ i)$ and $Y(\sigma,\ i)$ refer to the reserves of tokens X and Y after trade i, as follows:

$$X(\sigma, i) = \begin{cases} x_0 & \text{if } i = 0 \\ X(\sigma,\ i-1) + x_{\text{in}} & \text{if } i > 0 \text{ and } \sigma(i) = \text{ sell } x_{\text{in}} \\ X(\sigma,\ i-1) - x_{\text{out}}(\sigma, i) & \text{if } i > 0 \text{ and } \sigma(i) = \text{ buy } y_{\text{in}} \end{cases} \tag{5}$$

$$Y(\sigma, i) = \begin{cases} y_0 & \text{if } i = 0 \\ Y(\sigma,\ i-1) + y_{\text{in}} & \text{if } i > 0 \text{ and } \sigma(i) = \text{ buy } y_{\text{in}} \\ Y(\sigma,\ i-1) - y_{\text{out}}(\sigma, i) & \text{if } i > 0 \text{ and } \sigma(i) = \text{ sell } x_{\text{in}} \end{cases} \tag{6}$$

C Sandwich Attacks

The CLVR algorithm prevents 3-transaction sandwich attacks. In this section we provide a proof.

The three transactions involved in a sandwich attack are (1) the *front-running* transaction, (2) the *victim* transaction, and (3) the *back-running* transaction. The front-running and back-running transactions are inserted by an attacker in order to manipulate the victim into buying at an inflated price. The attacker is thus able to extract risk-free profit.

In an attack, the transactions execute in exactly the order listed above. In order for this manipulation to be profitable, the front-running transaction must push the price toward one less favorable for the victim, thus causing them to overpay. Finally, the back-running transaction swaps in the opposite direction in order to recover a risk-free profit denominated in the token originally sold to the AMM.

Without loss of generality (see Section 4.1), consider that the victim transaction sells token X to the AMM in order to receive token Y. Then, the front-running transaction also sells X to receive Y, and the back-running transaction sells Y to receive X.

We define the following variables. The amounts paid to the AMM by the front-running, and back-running transactions are Δx_f and Δy_b, respectively, and the amounts received from the AMM are Δy_f and Δx_b, respectively (the victim transaction size is not important for the proof). Let x_0 and y_0 be the liquidity pool's reserves of token X and token Y, respectively, before transactions execute. Let $p_0 = y_0/x_0$ be the initial price.

The attacker receives a risk-free profit when $\Delta x_f < \Delta x_b$ and $\Delta y_b \leq \Delta y_f$ – i.e., when the front-running transaction pays less X than the back-running transaction receives, *and* the attacker's back-running transaction does not spend more token Y than the attacker received when executing the front-running transaction. The former condition guarantees profit, and the latter condition guarantees that it is risk-free – i.e., the attacker does not lose any token Y.

Now, our task is determine whether it is possible to perform a risk-free sandwich attack when also subject to the CLVR sequencing rule. We proceed by contradiction. Start by considering how to sequence the three transactions involved in the sandwich attack. According to the sequencing rule, the first transaction must move the log-price the least.

Let p_f and p_b be the prices after executing the front-running and back-running transactions first (before any other transactions execute), respectively. scThen we must ensure that $(\ln p_0 - \ln p_f)^2 \leq (\ln p_0 - \ln p_b)^2$ (or else the back-running transaction would be sequenced first, violating the sandwich attack order). Since we know that $p_f < p_0 < p_b$ (given the trade directions), it must be that $\ln p_0 - \ln p_f \leq \ln p_b - \ln p_0$ or, equivalently, $p_0^2 \leq p_b p_f$, which is expressed as:

$$\left(\frac{y_0}{x_0}\right)^2 \leq \left(\frac{\Delta y_b + y_0}{-\frac{\Delta y_b x_0}{\Delta y_b + y_0} + x_0}\right)\left(\frac{-\frac{\Delta x_f y_0}{\Delta x_f + x_0} + y_0}{\Delta x_f + x_0}\right) = \frac{(\Delta y_b + y_0)^2}{(\Delta x_f + x_0)^2} \tag{7}$$

$$\tag{8}$$

Since all values are positive,

$$\frac{y_0}{x_0} \leq \frac{\Delta y_b + y_0}{\Delta x_f + x_0} \tag{9}$$

Then, we must also satisfy $\Delta y_b \leq \Delta y_f$ (the back-running transaction cannot spend more Y than the front-running transaction purchases, or else violate risk-free profit). Expanding Δy_f yields:

$$\Delta y_b \leq \frac{\Delta x_f y_0}{\Delta x_f + x_0} \tag{10}$$

Combining Inequality 9 and Inequality 10 gives:

$$\frac{y_0}{x_0} \leq \frac{\frac{\Delta x_f y_0}{\Delta x_f + x_0} + y_0}{\Delta x_f + x_0} \tag{11}$$

$$\frac{y_0}{x_0} \leq \frac{y_0\left(2\Delta x_f + x_0\right)}{\left(\Delta x_f + x_0\right)^2} \tag{12}$$

$$\left(\Delta x_f + x_0\right)^2 \leq x_0\left(2\Delta x_f + x_0\right) \tag{13}$$

$$\Delta x_f^2 + 2\Delta x_f x_0 + x_0^2 \leq 2\Delta x_f x_0 + x_0^2 \tag{14}$$

$$\Delta x_f^2 \leq 0 \tag{15}$$

which is a contradiction, as Δx_f is a positive real value.

D On the Quality of CLVR:

A natural question to ask is whether, given a block of transactions, the CLVR rule always achieves the globally minimal price volatility. Unfortunately, we can find a counter examples showing it does **not**. For example, consider three trades: $\{\alpha:\ \text{sell 2},\ \beta:\ \text{sell 5},\ \gamma:\ \text{buy 10}\}$ when the initial liquidity available is $x_0 = y_0 = 100$. The optimal ordering for minimizing aggregate price-volatility is **not** to start with sell 2, as CLVR would dictate. Rather, it is sell 5, buy 10, sell 2. The CLVR ordering is illustrated in Figure S1 (right) against the optimal volatility-minizing ordering (left). In this case, the average price volatility of the CLVR is 8.2×10^{-3}, while the optimal ordering can reduce this volatility to 7.9×10^{-3}.

This example highlights why CLVR only approximately minimizes price volatility. However, even in this example, we can still see that the CLVR order gives very low price volatility, which is nonetheless close to optimal. In section 5.2, we provide evidence that CLVR achieves nearly-optimal orderings in many cases.

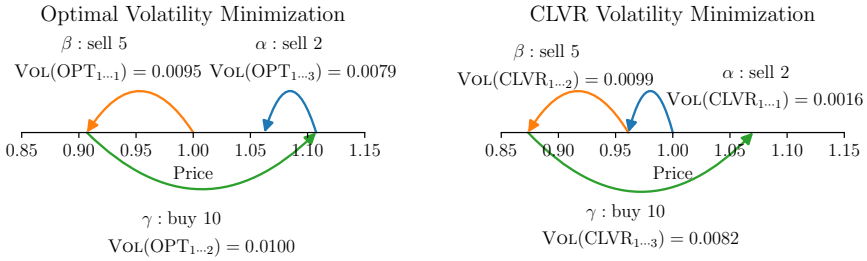


Figure S1. Optimal sequence minimizing price volatility, OPT (left) vs CLVR sequence (right)

E Robustness checks

In this section, we run some robustness checks. We (1) change the liquidity available in the pool, and (2) change the size distribution from log normal to uniform.

Table S2. Comparison of VHGSR vs CLVR. When tractable, relative volatility is computed as a percentage where 0% is the minimum volatility, and 100% is the maximum. The p-value is computed from a paired, one-sided t-test.

Lower Liquidity $x_0 = y_0 = 100,000$						
	Winner Count			Mean Relative Volatility		
Block Size	VHGSR	CLVR	Tie	VHGSR	CLVR	p-value
3	0	18	82	8.55	0.27	0.078
5	0	58	42	10.14	0.19	0.017
10	3	89	8	2.96	0.08	0.009
12	3	93	4	1.74	0.07	0.021
13	4	88	8	2.36	0.09	0.038
Uniform Distribution, trade size selected uniformly on $(0,\ 100,000)$						
	Winner Count			Mean Relative Volatility		
Block Size	VHGSR	CLVR	Tie	VHGSR	CLVR	p-value
3	0	9	91	3.12	0.20	0.004
5	1	37	62	2.78	0.67	< 0.001
10	6	61	33	0.75	0.40	< 0.001
12	8	76	16	0.60	0.30	< 0.001
13	11	73	16	0.43	0.27	< 0.001

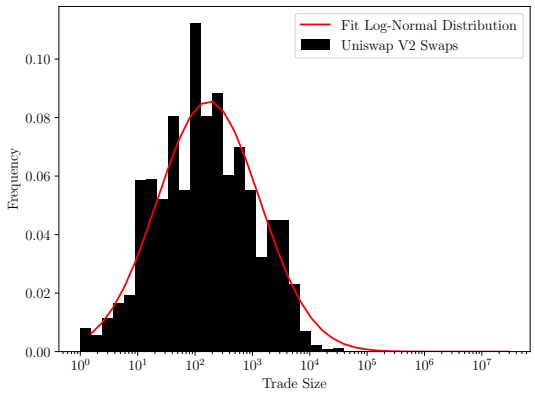


Figure S2. Distribution of 4,192 swaps executed on Uniswap V2's USDC–USDT AMM between June 19, 2024 and July 19, 2024 (UTC) . The figure also illustrates the goodness of fit of the swap size distribution to a log normal distribution with parameters $\mu = 4.93$ and $\sigma = 2.05$.

F Splitting

Traders may split their transactions into small trades, also known as order splitting [25], to strategically improve their exchange prices and avoid price manipulations. In addition, splitting transactions into smaller ones provides more flexibility with respect to ordering possibilities, which helps improve the system's price stability. However, transaction splitting also entails costs, since traders must pay gas fees for each transaction. As a result, splitting incurs significantly higher gas costs than executing a single transaction, as we explain further in Section N. Here, we focus on the benefits of transaction splitting for execution and price stability.

We can explain this idea with the following example: suppose there are only two pending transactions in a block. One is buying (α : buy $1,000$), and the other is selling (β : sell $1,000$). Suppose that $x_0 = y_0 = 100,000$ and, thus, $p_0 = 1$. In this example, if we execute both transitions, we will have a price volatility of 2.0×10^{-4}. However, if both traders split their transactions into 1000 small transactions of 1 each, an optimizing algorithm can improve the price volatility to 2.0×10^{-10} – an improvement of 6 orders of magnitude. The optimal ordering interleaves smaller buy and sell orders, alternating between each, in order to maintain the status-quo price. The more users split their transactions, the more this price volatility will decrease and eventually converge to 0.

The social planner may want to encourage order-splitting, especially when processing large transactions with significant price impact.[2] We perform the following two experiments to test whether traders can benefit from splitting their transactions under CLVR. First, we check if a trader can expect better execution (in expectation) when splitting the transaction into two or more. We start by generating a 'buy' trade of size s, which we will refer to as t^*. Then, we

[2] In an environment where network(gas) fees need to be paid for each transaction's execution, splitting the transactions may have boundaries that both traders and the social planner will need to take into account.

generate 9 additional trades randomly, according to the log-normal distribution described above. We sequence these 10 trades using CLVR, and compute the amount of token received by t^*. Then, we split t^* into n equally-sized trades $t_1^*, \cdots, t_n^*$, and again sequence the trades and sum the amount of token received by each trade t_i^*. This process is repeated 1,000 times for each combination of transaction size s and number of splits n. The results of this experiment are drawn in Figure S3. We find that for all 10 trade sizes tested (varying from 10 to 10 million), the trader experiences a slight increase in amount out, on average, which is increasing in the number of splits n.

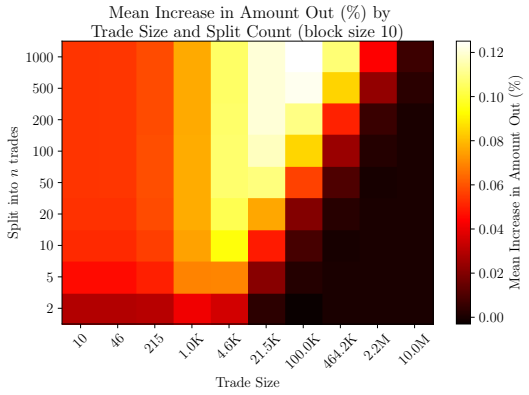


Figure S3. Trade-splitting experiment outcome. For various trade sizes, we compare the amount out (without splitting) vs the amount out after splitting the single trade n equally-sized trades.

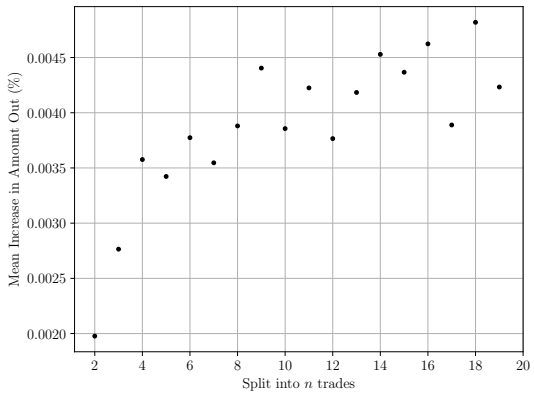


Figure S4. Trade-splitting experiment outcome (every trader splits). We provide evidence the mean percent increase in amount out when every trader splits into n trades.

Next, we examine what would happen to the ecosystem if *all* traders split their transactions given the benefits that we show in Figure S3. We perform the following experiment. First, we generate 10 trades at random using the log-

normal distribution described above. Then, we sequence the trades using CLVR and compute the amount out for each trade. Finally, we split each trade into n equally-sized trades, sequence them using CLVR, and again compute the amount out that each original trader would receive from their several trades. The process is repeated 1,000 times. We plot the results in Figure S4. We find an average increase in amount out, which increases in the number of splits n. To conclude, if one trader splits her transaction (no matter the transaction size), she can get better execution (in expansion) by splitting the transaction. Therefore, if all traders adopt this strategy, we show that they achieve better execution by splitting their transactions. This suggests that under CLVR, traders can benefit from splitting their transactions and thus have an incentive to do so.

G Computational Complexity of CLVR

In the main body of this work we present CLVR as an algorithm with $O(n^2)$ complexity: a simple loop-within-loop construction to minimize volatility at each step. In this section we describe a simple modification to achieve $O(n \cdot \log\ n)$ performance.

Intuitively, this modification relies upon the fact that at each step CLVR selects the next trade which minimizes an objective function: the single-step contribution to volatility. In the main body of this work we present this selection process as a simple linear scan over the trades which are not yet sequenced. Suppose, instead, that we begin by dividing the trades by their direction (buy / sell), and then sorting each list in a sorted collection keyed by the trade's size. Then, at each step, we are able to perform a selection from each collection via binary-search. The following contains a high-level sketch.

Consider an input list of trades and an AMM with a current price p_0. Divide the trades into balanced binary trees `trades_buy` and `trades_sell`, keyed by their quantity of assets traded. This consumes $O(\log\ n)$ computation in upfront preparation. For each step i, repeat the following process.

Suppose at step i the current price is p_i. Compute the ideal trade direction and volume which would return the price to p_0 (the set-point) – this is an $O(1)$ operation. Intuitively, the best trade to sequence will have a quantity which is most close to the computed ideal. We now can search for (at most) three candidate trades. From the list of trades in the ideal direction, use the ideal quantity to lookup the nearest trades, both larger and smaller in quantity, if present. This is $O(\log\ n)$. From the list of trades in the opposite direction, lookup the smallest such possible trade, if present. This is another $O(\log\ n)$. Among these (at most) three candidates, locate the ideal trade and select it. This is $O(1)$. Finally, delete the trade from the balanced binary tree, this is $O(\log\ n)$.

Thus, we have reduced the inner loop's complexity to $O(\log\ n)$ for each of n steps, and CLVR executes in $O(n \cdot \log\ n)$ time.

H Insensitivity of the Volatility Measure to Quote Direction

Below is a brief proof of the insensitivity to quote direction. In a nutshell, this is equivalent to choosing a unit of account, i.e., whether to quote the USD-JPY exchange rate in dollars per yen, or yen per dollar. This becomes important when measuring how much the price has deviated from the starting point. A naïve approach would be to simply compute the absolute difference, however this makes the computation sensitive to which way the spot price is quoted. Put explicitly,

$$\left(\frac{\alpha_0}{\beta_0} - \frac{\alpha_0 + \Delta\alpha}{\beta_0 - \Delta\beta}\right)^2 \neq \left(\frac{\beta_0}{\alpha_0} - \frac{\beta_0 - \Delta\beta}{\alpha_0 + \Delta\alpha}\right)^2 \tag{16}$$

CLVR's computation is insensitive to this choice, as it considers instead the absolute difference in *log* prices, thus making the computation insensitive to the quote direction. We prove this statement below.

$$\left(\ln\frac{\alpha_0}{\beta_0} - \ln\frac{\alpha_0 + \Delta\alpha}{\beta_0 + \Delta\beta}\right)^2 = \left(-\left(\ln\frac{\alpha_0}{\beta_0} - \ln\frac{\alpha_0 + \Delta\alpha}{\beta_0 + \Delta\beta}\right)\right)^2 \tag{17}$$

$$= \left(\left(-\ln\frac{\alpha_0}{\beta_0}\right) - \left(-\ln\frac{\alpha_0 + \Delta\alpha}{\beta_0 + \Delta\beta}\right)\right)^2 \tag{18}$$

$$= \left(\ln\left[\left(\frac{\alpha_0}{\beta_0}\right)^{-1}\right] - \ln\left[\left(\frac{\alpha_0 + \Delta\alpha}{\beta_0 + \Delta\beta}\right)^{-1}\right]\right)^2 \tag{19}$$

$$= \left(\ln\frac{\beta_0}{\alpha_0} - \ln\frac{\beta_0 + \Delta\beta}{\alpha_0 + \Delta\alpha}\right)^2 \tag{20}$$

I An Additional Example of Ordering Effects

The following example illustrates a situation where transaction ordering significantly impacts execution price. Let $x_0 = y_0 = 100$, then $k = x_0 y_0 = 10,000$. Now, consider a set of two pending transactions $\{\alpha:\ \text{sell } 10,\ \beta:\ \text{sell } 10\}$, which we order as $\sigma:\ \{1 \mapsto \alpha,\ 2 \mapsto \beta\}$. By applying Equation 2, we compute the amount of token received for each transaction: $y_{\text{out}}(\sigma, 1) = 9.09$, $y_{\text{out}}(\sigma, 2) = 7.58$. Observe that despite each trader spending the same quantity of token X (10), they each receive significantly different amounts of token Y, depending on execution order.

In the above example, execution ordering impacts which trader receives 9.09 units of Y, and which trader receives 7.58 units of Y.

J Alternative Economic Goals

Price stability is not the only economic goal that a social planner may want to achieve. In Appendix A we examine an alternate goal: reducing inequality. In a nutshell, a social planner that is minimizing inequality seeks to give "good" prices to the less-wealthy traders at the expense of wealthy traders. We show that this goal is incompatible with maintaining price stability – they cannot be simultaneously pursued. Further frustrating this objective is the fact that inequality is a somewhat illusive concept on blockchains, since wallet ownership is anonymized and multiple wallets can be under the control of a single user (a *sybil* attack). It is left for future research to further investigate the impact of transaction orderings on inequality, coupled with the challenge of analyzing wealth ownership on blockchains.

K Effect of Liquidity on CLVR's Worst-Case Performance

Finally, given the CPMM pricing rule, we know that another important factor influencing price volatility is the transaction size relative to the available liquidity in the pool, as explained in [9]. When the ratio of transaction size to pool liquidity increases, the CPMM pricing rule yields a higher execution price, and this ratio can therefore affect the optimality of CLVR when it varies.

To explore this dimension, we select several specific sets of pending transactions in which CLVR exhibits relatively high price volatility (i.e., poor performance) and examine how relative price volatility changes as we vary the liquidity available in the pool. Figure S5 presents the results for a representative example. The figure suggests that while reducing liquidity generally increases price volatility, it can also improve CLVR's relative performance: the CLVR algorithm tends to produce outcomes closer to the optimal ordering rather than to the maximum volatility, implying that CLVR performs well under the relative volatility metric. In contrast, increasing the liquidity in the pool appears to have only a marginal impact on CLVR's relative volatility.

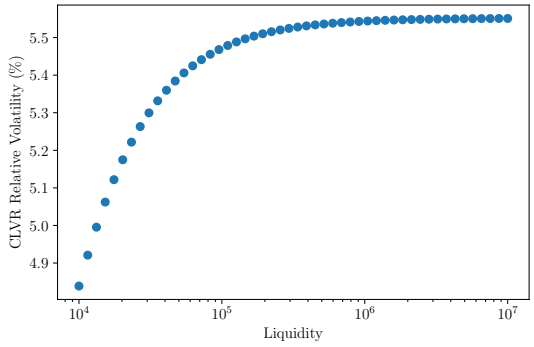


Figure S5. The y-axis represents CLVR's relative volatility for the worst-case set of pending transactions identified in Table 1, while the x-axis shows how price volatility changes as the amount of liquidity in the pool varies for this specific set. Liquidity is measured as $x_0 = y_0$, the reserves of the AMM.

L Additional Related Work

Another way is to protect traders by setting a slippage tolerance, which sets a limit price above which a trade will not execute [10,17]. While this approach can protect traders from price manipulation and sandwich attacks, it does not contemplate the economic good offered by imposing an ordering mechanism, as we offer in this work.

An alternative way to protect users is to generate an auction between the traders to determine priority of their transactions [12]. Still, this approach will not leverage the main benefit of ordering a transaction to optimally achieve economic goals such as reducing price deviation or inequality. One way to address this issue is to offer batch auctions, which first intended to mitigate high-frequency trading on traditional markets[6]. The main idea of this approach is to collect all pending transactions and run an auction to determine a single price that clears the market in a way that does not give any particular trader a better exchange price relative to another. [26,28,7] offer models to implement this idea in AMMs. Our approach is similar to the batch auction in that it leverages the idea of executing transactions together to clear the market better and provide better prices. However, we offer to achieve this goal on an AMM by implementing an ordering mechanism that can be used without modifying the particular mechanics of how an order is filled.

Finally, previous work has explored goals other than price stability in AMMs, such as comparing them to centralized exchanges and limit order book mechanisms, and examining the risks and benefits of liquidity provision in these markets, and more [3,20,15,13,24,16,8,22,23,1,2]. Our main contribution to this area of research is to shed light on how an AMM can improve price stability while imposing a transaction ordering mechanism.

M Price Stability and Block Sizes

The next set of experiments examines the impact of block size on price stability under CLVR ordering. In traditional financial markets, there is an extensive discussion on what the optimal trading frequency is [11,5,19,14].

Our evaluation strategy is as follows. We synthesize a set of 100 transactions using the process described above (in section 5.1). Each transaction is given a sequential timestamp, which indicates the order of the transaction's arrival in the "mempool". For simplicity, we will assume that the spot market price is equal to 1, as we set in the previous simulations.

Now we can test what will happen if the social planner executes all the transactions using a first come, first served policy, which can be seen as having 100 blocks (one transaction per block). We also examine what would happen if there were batches of different block sizes, for example, ten blocks with ten transaction each, or one big block with 100 transactions. We test this over a variety of block sizes.

Figure S6 shows the results of running this simulation over 1,000 trials, demonstrating the advantage in price stability with batching transactions in

larger blocks. When we have a first come, first serve approach (on the left), this gives the highest median price volatility with a very high variance. In contrast, price volatility is significantly reduced when all transactions are batched into one big block (on the right). This allows the CLVR ordering mechanism to achieve a much lower median of price volatility with much lower variance. In summary, transaction ordering has a significant impact on price volatility when the block size is very large. A social planner may want to take this into consideration when deciding the trading frequency.

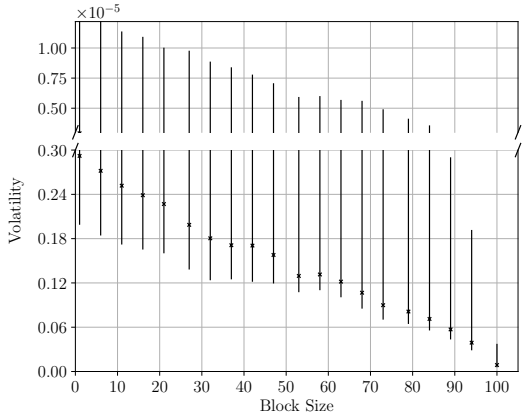


Figure S6. Price volatility achieved by CLVR with varying block sizes. Computed over 1,000 trials, error bars drawn at 25th and 75th percentiles, marker placed at median.

N Limitations and Future Directions

Practical trading aspects. We do not account for traders using routers, which split transactions over several liquidity pools – this would require a multi-pool analysis. **Protocol complexities.** If the social planner is able to censor or insert transactions, there may be implications on iterated (multi-block) sequencing which circumvent CLVR's ordering capabilities. There is promising concurrent work on anti-censorship measures, such as inclusion lists, which tackle this problem [21,18,4]. **Gas fees.** We also do not account for a protocol's gas prices and fees when considering a trader's decision to split their transaction. Transaction splitting can also be viewed as equivalent to increasing block size, in the sense that once transactions are split into many smaller pieces, the effect is similar to increasing the number of transactions per block. Future work may examine the impact of fees on traders' strategic decision-making and also the impact on the block size. **Future directions.** Transaction ordering may have an impact on a market's price discovery over time, especially if a social planner decides to increase the block-time. Enforcing an ordering mechanism may also have implications for the activities of informed traders (e.g., centralized exchange arbitrageurs). Future work should analyze the impact of CLVR on liquidity providers and how it mitigates Loss-Versus-Rebalancing (LVR) [24].

References

1. Angeris, G., Chitra, T.: Improved price oracles: Constant function market makers. In: Proceedings of the 2nd ACM Conference on Advances in Financial Technologies. pp. 80–91 (2020)
2. Angeris, G., Chitra, T., Evans, A.: When does the tail wag the dog? curvature and market making (2022)
3. Aoyagi, J., Ito, Y.: Coexisting exchange platforms: Limit order books and automated market makers (2021)
4. Babel, K., Jean-Louis, N., Ji, Y., Misra, U., Kelkar, M., Mudiyanselage, K.Y., Miller, A., Juels, A.: Prof: Protected order flow in a profit-seeking world. arXiv preprint arXiv:2408.02303 (2024)
5. Budish, E., Cramton, P., Shim, J.: Implementation details for frequent batch auctions: Slowing down markets to the blink of an eye. American Economic Review **104**(5), 418–424 (2014)
6. Budish, E., Cramton, P., Shim, J.: The high-frequency trading arms race: Frequent batch auctions as a market design response. The Quarterly Journal of Economics **130**(4), 1547–1621 (2015)
7. Canidio, A., Fritsch, R.: Arbitrageurs' profits, lvr, and sandwich attacks: batch trading as an amm design response. arXiv preprint arXiv:2307.02074 (2023)
8. Capponi, A., Jia, R.: The adoption of blockchain-based decentralized exchanges. arXiv preprint arXiv:2103.08842 (2021)
9. Chemaya, N., Liu, D.: Estimating investor preferences for blockchain security. Available at SSRN 4119827 (2022)
10. Chemaya, N., Liu, D., McLaughlin, R., Ruaro, N., Kruegel, C., Vigna, G.: The power of default: Measuring the effect of slippage tolerance in decentralized exchanges. Available at SSRN 4585229 (2023)
11. Du, S., Zhu, H.: What is the optimal trading frequency in financial markets? The Review of Economic Studies **84**(4), 1606–1651 (2017)
12. Ernst, T., Spatt, C.S., Sun, J.: Would order-by-order auctions be competitive? Available at SSRN 4300505 (2022)
13. Exchanges, C.V.D., Barbon, A., Ranaldo, A.: On the quality of cryptocurrency markets (2022)
14. Fricke, D., Gerig, A.: Too fast or too slow? determining the optimal speed of financial markets. Quantitative Finance **18**(4), 519–532 (2018)
15. Harvey, C.R., Hasbrouck, J., Saleh, F.: The evolution of decentralized exchange: Risks, benefits, and oversight. Benefits, and Oversight (June 15, 2024) (2024)
16. Hasbrouck, J., Rivera, T.J., Saleh, F.: The need for fees at a dex: How increases in fees can increase dex trading volume. Available at SSRN 4192925 (2022)
17. Heimbach, L., Wattenhofer, R.: Sok: Preventing transaction reordering manipulations in decentralized finance. In: Proceedings of the 4th ACM Conference on Advances in Financial Technologies. pp. 47–60 (2022)
18. https://ethresear.ch/: Unconditional inclusion lists. `https://ethresear.ch/t/unconditional-inclusion-lists/18500` (2024)
19. Jagannathan, R.: On frequent batch auctions for stocks. Journal of Financial Econometrics **20**(1), 1–17 (2022)
20. Lehar, A., Parlour, C.A.: Decentralized exchange: The uniswap automated market maker. Journal of Finance forthcoming (2021)
21. mike (@michaelneuder), (@vbuterin), V., (@fradamt), F., (@terencechain), T., potuz (@potuz), (@manav2401), M.: Eip-7547: Inclusion lists [draft]. `https://eips.ethereum.org/EIPS/eip-7547` (2023)

22. Milionis, J., Moallemi, C.C., Roughgarden, T.: The effect of trading fees on arbitrage profits in automated market makers. In: International Conference on Financial Cryptography and Data Security. pp. 262–265. Springer (2023)
23. Milionis, J., Moallemi, C.C., Roughgarden, T.: A myersonian framework for optimal liquidity provision in automated market makers. arXiv preprint arXiv:2303.00208 (2023)
24. Milionis, J., Moallemi, C.C., Roughgarden, T., Zhang, A.L.: Automated market making and loss-versus-rebalancing. arXiv preprint arXiv:2208.06046 (2022)
25. Park, A.: The conceptual flaws of decentralized automated market making. Management Science **69**(11), 6731–6751 (2023)
26. Ramseyer, G., Goyal, M., Goel, A., Mazières, D.: Augmenting batch exchanges with constant function market makers. arXiv preprint arXiv:2210.04929 (2022)
27. Yitzhaki, S., Schechtman, E.: The Gini methodology: a primer on a statistical methodology. Springer Series in Statistics, Springer, New York, NY (2013). `https://doi.org/10.1007/978-1-4614-4720-7`
28. Zhang, M., Li, Y., Sun, X., Chen, E., Chen, X.: Computation of optimal mev in decentralized exchanges. Working paper-https://mengqianzhang.github.io/papers/batch.pdf (2024)